Reliability of Large Language Models for Design Synthesis: An Empirical Study of Variance, Prompt Sensitivity, and Method Scaffolding

***Abstract*—Large Language Models (LLMs) are increasingly applied to automate software engineering tasks, including the generation of UML class diagrams from natural language descriptions. While prior work demonstrates that LLMs can produce syntactically valid diagrams, syntactic correctness alone does not guarantee meaningful design. This study investigates whether LLMs can move beyond diagram translation to perform design synthesis, and how reliably they maintain design-oriented reasoning under variation. We introduce a preference-based few-shot prompting approach that biases LLM outputs toward designs satisfying object-oriented principles and pattern-consistent structures. Two design-intent benchmarks, each with three domain-only, paraphrased prompts and 10 repeated runs, are used to evaluate three LLMs (ChatGPT 4o-mini, Claude 3.5 Sonnet, Gemini 2.5 Flash) across three modeling strategies: standard prompting, rule-injection prompting, and preference-based prompting, totaling 540 experiments (i.e. 2x3x10x3x3). Results indicate that while preference-based alignment improves adherence to design intent it does not eliminate non-determinism, and model-level behavior strongly influences design reliability. These findings highlight that achieving dependable LLM-assisted software design requires not only effective prompting but also careful consideration of model behavior and robustness.**

[1]

***Index Terms*—Large Language Models, UML Class Diagrams, Design Synthesis, Behavioral Reliability, Preference-Based Alignment, Software Engineering Automation.**

I. Introduction

Large Language Models (LLMs) are increasingly used to automate software engineering tasks [2]–[5], including generating UML class diagrams from natural language [6], [7]. While prior work shows that LLMs can produce syntactically valid diagrams, syntax alone does not ensure good design in terms of maintainability, extensibility, and architectural soundness. In practice, many LLM-generated diagrams remain structurally shallow, reflecting surface-level entities but failing to capture design intent—namely design principles (e.g., abstraction, encapsulation, separation of concerns) and design knowledge (e.g., GoF patterns) that experienced designers apply to support extensibility and reuse [6]–[8]. Without this, even small input variations can lead to inconsistent and unreliable outputs [4], [9], [10].

This limitation reflects a key distinction between diagram translation and design synthesis. Existing approaches often act as translators, mapping textual elements to modeling constructs. However, effective architectural design requires inferring implicit principles and patterns from domain requirements, which are rarely explicitly specified in real-world setting. Therefore, LLM-based modeling should be evaluated on their ability to infer and apply design knowledge, rather than simply reproducing patterns mentioned in prompts [11], [12].

Beyond design quality, an underexplored challenge is behavioral reliability. Although LLMs are inherently non-deterministic, architectural workflows require stable and repeatable outputs. Practitioners must expect consistent designs under variations such as prompt paraphrasing or repeated runs; otherwise, utility declines regardless of average performance. Despite increasing adoption, limited work has systematically examined the reliability of design-level reasoning in frontier models [13]. Existing studies emphasize capability while overlooking stability across runs, sensitivity to prompt variation, and differences across models [12]–[14]. For architecture-centric tasks, where inconsistencies can propagate into implementation defects, this gap is particularly critical.

This leads to the following research questions in this study. **RQ1:** To what extent can LLMs generate design-level UML class diagrams that satisfy domain constraints and design principles when provided

[1]For the reproducibility of the experiments conducted in this research, the data is available at [1]

with domain-only descriptions? **RQ2:** To what extent do LLM-generated diagrams exhibit pattern-consistent structures implicitly, and how does the proposed approach affect the emergence of such structures compared to standard prompting and rule-injection baselines? **RQ3:** How reliable are the generated designs across repeated runs and prompt paraphrases, and what behavioral stability differences emerge across models? **RQ4:** How does increasing domain complexity affect adherence to design principles, pattern emergence, and behavioral stability?

This paper investigates whether LLMs can be guided from translation toward design synthesis and how reliably they maintain design-oriented reasoning under controlled variation. We propose a preference-based few-shot prompting approach that exposes models to contrasting solutions and biases generation toward designs aligned with object-oriented principles and pattern-consistent structures[2]. To avoid trivialization, we construct two design-intent benchmarks where problem descriptions specify domain behavior without naming principles or patterns. We evaluate three LLMs (ChatGPT 4o-mini, Claude 3.5 Sonnet, Gemini 2.5 Flash) across three paraphrased prompts and three prompting strategies—standard, rule-injection, and preference-based—over 10 repeated runs (540 total experiments).

Our results show that performance differences in design synthesis cannot be attributed to prompting strategy alone; model-level behavior plays a critical role in reliability. While preference-based alignment improves adherence to architectural intent, distinct stability patterns emerge: Claude shows high consistency across runs, ChatGPT is predictable once design scaffolding is established, and Gemini exhibits significant variability across prompts and executions. These findings suggest that model choice is a dominant factor in achieving dependable software design, reframing LLM-based modeling as both a capability and reliability challenge.

Paper Structure: Section 2 reviews related work, Section 3 presents the experimental design, Section 4 describes the prompting strategies, Section 5 reports results and discussion, Section 6 outlines limitations, and Section 7 concludes the paper.

II. Background and Related Work

LLMs for Software Design Early studies showed that models such as Codex and GPT-3 can generate syntactically correct code from natural language, demonstrating their potential as programming assistants [16]. Subsequent work extended this to software design, exploring the generation of UML diagrams from text [6], [7], [17], [18] and suggesting support for early-stage modeling.

However, syntactic correctness does not ensure design quality [13], [14]. This reflects broader concerns about hallucinations in generative AI, where outputs may appear plausible but remain semantically inconsistent or incomplete with respect to domain requirements [7], [12], [19].

Design Quality and Pattern Consistency Studies on LLM-assisted modeling show that models often fail to internalize design knowledge, producing diagrams with weak hierarchies, poor encapsulation, and inconsistent patterns [6], [7], [11]. For example, [13] note that while LLMs can extract classes and relationships, they frequently misrepresent associations or omit behavioral constraints. This highlights the gap between diagram translation and true design synthesis.

Prompt Engineering and Preference-based Alignment Recent work addresses this gap through prompt engineering techniques such as chain-of-thought and few-shot learning to align model reasoning with object-oriented principles [8], [11], [20], as well as rule-based approaches to enforce design adherence [8], [21]. However, these methods often depend on domain-specific rules, limiting generalization to unseen scenarios.

More recent approaches move toward design synthesis using preference-based alignment and reinforcement learning from human feedback (RLHF). By exposing models to contrasting solutions, these methods encourage outputs that satisfy architectural principles and pattern consistency [22]. Applied to UML generation, they can guide models toward design-oriented outputs without explicitly specifying patterns, supporting more robust modeling.

Behavioral Reliability and Model Variability Beyond correctness, LLMs pose challenges in behavioral reliability. As stochastic systems, their outputs can vary across runs, prompt paraphrases, and model families [13], [14], undermining trust in AI-assisted workflows [10], [23]. Prior work shows that models exhibit distinct stability patterns, with some producing consistent outputs while others fluctuate significantly [12], [14]. This highlights the need to evaluate not only design fidelity but also reliability under realistic variations.

Perspective of current study: While LLMs can generate syntactically valid diagrams, achieving design synthesis requires addressing three key challenges: internalizing design principles, ensuring behavioral stability, and evaluating both correctness and reliability.

[2]A preliminary version appeared as an abstract-only workshop contribution [15]; this paper extends it with full implementation and comprehensive evaluation.

Principles and Patterns	Application in HMS Reference Model	Application in Sensor Reference Model
Abstraction [24], [25]	Patient, InPatient, OutPatient class	Sensor, PressureSensor, TemperatureSensor, RainGauge, WindSensor
Encapsulation [24], [25]	Private attributes used	Private attributes used
Observer Pattern [26], [27]	Not Applicable	Subject, Observer, WeatherInformationSystem, WeatherStation
Strategy Pattern [26]	BillingStrategy, BillingProcessor,InsuranceBilling, SelfPayBilling, GovernmentBillings	RepotingStrategy, ReportingContext, HourlyReporting, CustomIntervalReporting

Table I: Application of established design principles and patterns in Reference Models

Our study builds on this by examining how model choice and prompting strategies jointly affect the quality and robustness of generated UML diagrams.

III. Experimental Design

We evaluate three state-of-the-art models representing different architectures and decoding characteristics (i.e. ChatGPT 4o-mini, Gemini 2.5 Flash, Claude 3.5 Sonnet). These models are evaluated under three different approaches (i) standard prompting, (ii) rule-injected prompting, (iii) preference-based few-shot prompting, with three paraphrased prompts per benchmark, and a temperature value set to 0.5.

A. Experiment Conduction

To ensure realistic evaluation, we constructed two domain-focused benchmarks. A medium-complexity system describing policy-based behavior in a hospital billing system (HMS), and a high-complexity system modeling sensor networks with event-driven interactions. For each benchmark, prompts are phrased thrice without naming any design patterns or rules, ensuring that models must infer architectural structures implicitly.

Per benchmark, we ran 10 experiments per paraphrased prompt with each model (ChatGPT, Claude, Gemini) and each prompting methodology (standard, rule-injected, preference-based) collecting a total of 540 UML diagrams: 2 domains$\times 3$ prompts$\times 3$ models$\times$ 3 prompting approaches $\times$ 10 runs $=$ 540 outputs. We then evaluated each diagram for correctness, principles and pattern consistency, and stability against the reference model designed by an MDE focused Ph.D. student for each benchmark. To reduce evaluator bias, the reference model and rule mapping were independently reviewed by an additional modeling expert with industrial experience and the disagreements were resolved through discussion. From the reference models of these benchmarks we identified two design principles (i.e. abstraction and encapsulation), and two design patterns (i.e. Observer pattern, strategy pattern) in total. Table I lists these principles and patterns, their literature sources, and how they are instantiated in each reference model.

B. Evaluation Settings

We conduct a manual evaluation to better capture the architectural quality of generated models. This decision is motivated by recent work in LLM-based code and model generation, which just not only highlight the shortcomings of automated evaluation of the models [8] but also token-level metrics for measuring code or model generation quality [28] emphasizing the need for semantic, structural, and expert-informed evaluation methods that better reflect real design quality and usability [29], [30].

1) Evaluation Metrics: For each model, we define four metrics:

- **M1.1: Structural Correctness Score:** Measures the degree to which the structural elements in the generated diagram match expert reference model. *Computation:* Let TP be the number of correctly predicted elements, FP false positives, and FN false negatives. Then:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

  However, this metric is token-level and does not fully reflect design intent of the generated model. Hence, it is supplemented by semantic metrics defined below.
- **M1.2: Principle Adherence Score:** Measures the extent to which the diagram adheres to established OOAD principles identified in reference model. *Computation:* A separate checklist of design rules is used for each benchmark (see table. I). Each diagram is assessed for adherence to each rule:

$$\text{Principle Adherence Score} = \frac{\#\text{Rules Satisfied}}{\text{Total Rules Evaluated}}$$

  This assessment is conducted by the authors of this paper using the reference model and associated rule mapping on ternary scale (0=no, 0.5= partially, 1=yes).
- **M1.3: Pattern Emergence Score:** Measures whether the generated diagram instantiate a specific design pattern identified in reference model.

RQ	Evaluation Metric
RQ1	M1.2. Principle Adherence Score (PAS)
RQ2	M1.3. Pattern Emergence Score (PES)
RQ3	M1.4. Stability Index (SI)
RQ4	Comparative analysis of PAS, PES, and SI

Table II: Mapping of RQs to Evaluation Metrics

Each diagram is assessed for emergence of design pattern:

$$\text{Pattern Emergence Score} = \frac{\#\text{Pattern Applied}}{\text{Total Patterns Evaluated}}$$

This assessment is conducted by the authors of this paper using the reference model and associated rule mapping on ternary scale (0=no, 0.5= partially, 1=yes).

- **M1.4: Stability Index (SI):** Measures variance in diagrams generated across runs and prompt variations, capturing behavioral reliability. Higher SI indicates better consistency.
  Computation:

$$\text{SI} = \frac{1}{1 + \left(\sigma^2_{\text{prompt}} + \sigma^2_{\text{run}}\right)}$$

To ensure systematic alignment between research questions, evaluation criteria, and metrics, each RQ is explicitly operationalized as shown in table II.

IV. Prompting Methods

To understand the limitations of LLMs in generating design-level UML class diagram, we conduct a series of experiments using three different prompting methods.

A. Standard Baseline: Standard LLM Prompting

In this experimental condition, we evaluate the raw design-generation capability of the selected LLMs by providing only the domain-level design task within the prompt, without any additional rule injection or role based prompting. The objective of this setup is to observe how the models naturally interpret and respond to architectural modeling tasks when operating under minimal instruction. By isolating the task description from external constraints or injected design knowledge, this condition serves as a baseline for assessing whether LLMs inherently perform design synthesis or merely translate textual entities into structural elements. This configuration allows us to characterize the default modeling behavior of each LLM and establish a reference point for comparison against rule-injection and proposed preference-based prompting approaches.

B. Rule-injected Baseline: Prompting with Rule Injection

The purpose of this experiment was to analyze to what extent does the use of structured prompts with explicitly injected general design principles and expert advices can guide the LLMs to generate design-level UML class diagrams. Unlike most previous work, which implicitly assumes LLMs understand good design practices, this setup deliberately encodes those expectations into the prompt. This experiment has been done as a necessary control to test whether prior LLMs under performed due to lack of architectural exposure, rather than capability. The prompt passed to each model was divided into three structural components: Priming (get the LLM ready using role-play prompting and embed the established OOAD design principles derived from literature), Question (the core class diagram generation task), and Decorator (the expert guidance on formatting, relationship modeling, and a self-check phase)

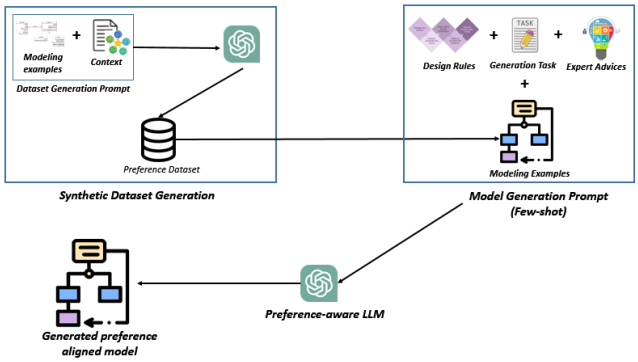

Figure 1: Proposed Approach

C. Proposed Approach: Preference-based Few-shot Prompting

In this section we present a preference-guided few-shot prompting approach. While our rule-injected baseline incorporated design rule prompts, this section introduces an additional guidance strategy, that in addition to design rule prompts, trains the LLM to recognize better and worse design alternatives through annotated preference-based examples. Of particular concern, it is not a full alignment pipeline like RLHF, instead it is a lightweight, empirical technique intended to explore whether preference signals shift the LLMs output in design domain towards architecturally sound structures.

1) Preference Dataset Generation: To generate our modeling dataset we carry out a whole new set of conversation with LLM (o1-mini) on good design principles and patterns. The aim of conducting this conversation was to provide enough modeling context to the LLM before it generates a cross-domain modeling dataset consisting of a set of five task description and a design solution. Since LLMs are not good at design task, the

generated diagram was also not up-to the mark and hence marked as a bad solution. The idea behind this decision was to let the LLM know that the way it generates diagram is not a preferred human choice. For a good design solution, we asked our modeling experts to define good design solutions against the generated problems. To be clear, all five data points in the dataset include a design-level description, two generated class diagram solutions (solution1: by LLM, solution2: by modeling experts), and preferred choice labeled by a human annotator (i.e. the expert generated model). The preference was manually annotated by the authors of this paper based on established OOAD principles and then validated by one modeling expert from industry.

It is to be noted that the benchmarks used in this study (i.e. HMS and Sensor Network) to evaluate the LLMs behavioral stability and reliability for design synthesis are not part of this dataset. Those benchmarks only contain the design description and one expert designed reference model to facilitate evaluation.

V. Results and Discussion

This section presents the findings of the study organized around the research questions. Rather than evaluating design generation purely as a capability problem, the analysis focuses on design quality, behavioral stability, and the conditions under which LLMs can reliably support architectural modeling.

A. Answer to RQ1

Across both domains and all evaluated models, the preference-aware approach consistently produced diagrams that more closely aligned with the reference architectures. Models operating under this setting applied a greater proportion of expected object-oriented principles (see table V). These results suggest that preference signals can guide models beyond surface-level entity extraction toward more deliberate architectural organization.

Among the three models, GPT demonstrated the strongest adherence to design principles in the preference-based setting, applying the highest number of architectural constructs present in the reference models, however, occasionally produced duplicate inheritance relationships and demonstrated partial constraint satisfaction. Claude generated structurally coherent diagrams but failed to abstract shared behavior into superclasses. Gemini showed comparatively weaker performance, often messing with key structural elements required to satisfy domain constraints including duplicate and circular relationships, conflicting inheritance hierarchies, and missing classes.

Minimal differences were observed between standard prompting (see table III) and rule injection for most models (see table IV), indicating that explicitly encoding design rules does not necessarily translate into improved architectural reasoning. Notably, Gemini's performance degraded under rule injection, potentially due to the increased cognitive load imposed by dense instructions, which appeared to trigger hallucinated relationships and inconsistent structures.

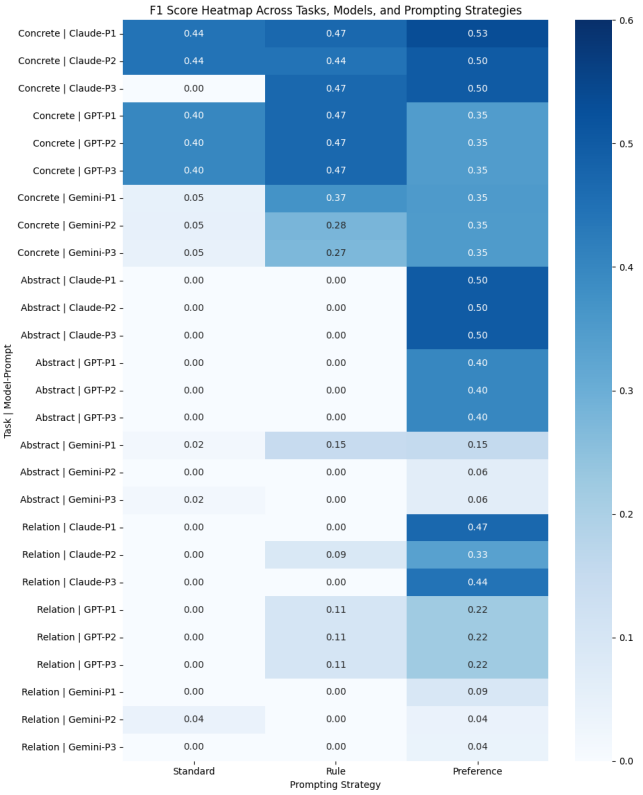

Figure 2: M1.1: HMS Performance Averaged across Runs, Models, and Prompting Strategies

Key Insight: LLMs may generate design-level diagrams when guided appropriately, but preference-based prompting appears more effective than prescriptive rule injection in supporting architectural reasoning.

B. Answer to RQ2

The preference-based approach increased the likelihood that models would produce pattern-consistent structures without explicitly naming patterns in the prompt (see table VI-VIII). This indicates that exposure to contrasting architectural solutions can bias models toward reusable design strategies rather than direct textual translation.

However, recurring design smells reveal that implicit pattern application remains incomplete. Claude frequently omitted aggregation relationships between the Context class and Strategy interface. GPT occasionally

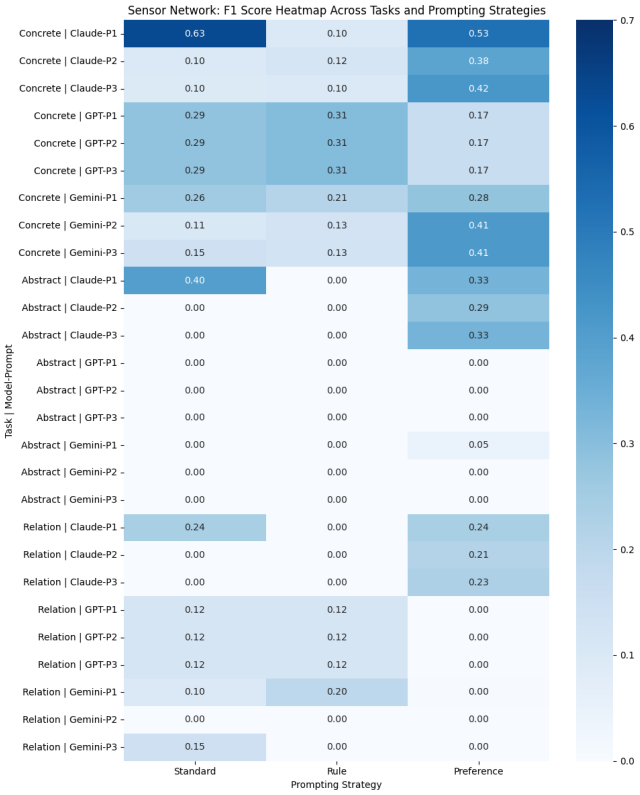

Figure 3: M1.1: Sensor Network Performance Averaged across Runs, Models, and Prompting Strategies

Task	Principle	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Abstraction	0	0	0	0	0	0	0	0	0
	Encapsulation	0	0	0	1	1	1	1	0	1
Sensor Network	Abstraction	0	0	0	0	0	0	1	0	0
	Encapsulation	0	0	0	0.5	0.5	0.5	1	1	1

Table III: M1.2: Principle Adherence Score — Standard Baseline Averaged across 10 runs

produced duplicate inheritance relationships. Gemini exhibited the highest concentration of structural issues, including conflicting inheritance hierarchies, and missing classes.

These findings suggest that while models can approximate pattern structures, they often struggle with enforcing the full set of architectural constraints required for correct implementation.

Key Insight: Preference optimization encourages pattern emergence, but current models still lack the global reasoning necessary to consistently operationalize design patterns. graphicx multirow

graphicx multirow

Task	Principle	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Abstraction	0	0	0	0	0	0	0	0	0
	Encapsulation	1	1	1	1	1	1	1	1	1
Sensor Network	Abstraction	0	0	0	0	0	0	0	0	0
	Encapsulation	0	0	0	1	1	1	1	1	1

Table IV: M1.2: Principle Adherence Score — Rule-injected Baseline Averaged across 10 runs

Task	Principle	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Abstraction	0.5	0.5	0.5	0	0	0	0	0	0
	Encapsulation	1	1	1	1	1	1	1	1	1
Sensor Network	Abstraction	0	0	0	1	1	1	0.5	0.5	0.5
	Encapsulation	1	1	1	1	1	1	1	1	1

Table V: M1.2: Principle Adherence Score — Preference-Based Approach Averaged across 10 runs

C. Answer to RQ3

A central contribution of this study is the identification of substantial reliability differences across frontier LLMs (see table IX). Claude produced varied architectures across approaches and paraphrased prompts but remained highly consistent across repeated runs for the same prompt. This behavior suggests strong decoding stability: once conditioned, the model tends to converge toward a similar structural solution. However, stability did not guarantee correctness, as several diagrams consistently omitted critical architectural relationships (see results from M1.1). GPT showed sensitivity to the modeling approach but was largely unaffected by paraphrasing or repeated executions. Once appropriate architectural scaffolding was established, the model behaved predictably, indicating a high degree of controllability. Importantly, GPT's errors appeared systematic rather than stochastic, suggesting bounded reasoning limitations rather than instability. Gemini, in contrast, exhibited significant variability across both runs and prompts, pointing to elevated stochasticity in structural generation. Although semantic intent often remained similar, diagram topology frequently shifted — with fluctuating numbers of classes and relationships — indicating architectural drift rather than deliberate refinement (see figures 2 and 3).

An important observation is that semantic consistency did not imply structural consistency. For architecture-centric tasks, structural variance is particularly consequential because inconsistencies may propagate into downstream implementation defects.

Key Insight: Reliability is strongly model-dependent. Model choice may therefore be as critical as prompting strategy when deploying LLMs for architectural workflows.

Task	Patterns	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Strategy Pattern	0	0	0	1	1	1	0	0	0
Sensor Network	Strategy Pattern	0	0	0	0	0	0	0	0	0
	Observer Pattern	0	0	0	0	0	0	0	0	0

Table VI: M1.3: Pattern Emergence Score — Standard Baseline Averaged across 10 runs

Task	Patterns	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Strategy Pattern	0	0	0	1	0	0	0	0	0
Sensor Network	Strategy Pattern	0	0	0	0	0	0	0	0	0
	Observer Pattern	0	0	0	0	0	0	0	0	0

Table VII: M1.3: Pattern Emergence Score — Rule-injected Baseline Averaged across 10 runs

D. Answer to RQ4

When task complexity increased from medium-scale problems to the high-complexity sensor network scenario, performance declined across all models — including under the preference-based setting. Most notably, no model successfully inferred the need for the Observer pattern in an event-driven architecture (see tables VI-VIII).

The Observer pattern requires recognizing implicit behavioral dependencies rather than explicit structural cues. The inability to detect this pattern suggests that current models rely heavily on pattern salience and struggle when architectural solutions must be inferred from dynamic system behavior.

This degradation indicates a ceiling effect in current design reasoning capabilities: as domains grow more complex and patterns become less obvious, models require stronger guidance or more advanced alignment mechanisms.

Key Insight: Architectural inference — particularly for behavior-driven patterns — remains a major open challenge for LLM-based design systems.

VI. Limitations

This study is preliminary and limited to two domains, and single modality generation tasks; design principles used as benchmark for measuring diagram quality; relies on expert labeling; and specific prompt design. It evaluates LLMs on design task using a small synthetic preference dataset before implementing a computationally costly, full RLHF-like pipeline. A broader validation of proposed approach on various domains, and in multi-artifact setup is left to future work.

VII. Conclusion and Future Work

This study systematically examined the capability and reliability of LLMs in generating UML class diagrams from domain-only descriptions, highlighting the distinction between diagram translation and true design synthesis, which requires reasoning about latent constraints, abstractions, and architectural patterns. Across 540 controlled experiments involving three LLMs, three prompting strategies, and multiple prompt paraphrases, we found that model-level behavior strongly affects design reliability. Preference-based few-shot alignment improved adherence to design principles and pattern-consistent structures, but non-determinism persisted, especially in complex domains.

Our findings suggest that achieving dependable LLM-assisted software design requires not just effective prompting, but careful model selection and robustness-aware methods, with behavioral stability treated as a core evaluation criterion alongside syntactic correctness and pattern adherence.

Future work will explore hybrid approaches combining preference-based prompting with explicit architectural reasoning, expand benchmarks to more complex domains, and investigate techniques—such as fine-tuning, constrained decoding, and ensembles—to reduce non-determinism in LLM outputs.

Task	Principle	ChatGPT			Gemini			Claude		
		P1	P2	P3	P1	P2	P3	P1	P2	P3
HMS	Strategy Pattern	0.5	0.5	0.5	1	1	1	1	1	1
Sensor Network	Strategy Pattern	0.5	0.5	0.5	0	0	0	0	0	0
	Observer Pattern	0	0	0	0	0	0	0	0	0

Table VIII: M1.3: Pattern Emergence Score — Preference-Based Approach Averaged across 10 runs

Model	Standard Baseline	Rule Baseline	Preference Prompting
Claude	0.77	1	1
ChatGPT	1	1	1
Gemini	0.905	0.905	0.907

Table IX: M1.3: Stability Index

VIII. Acknowledgment

The authors acknowledge the use of GenerativeAI tools, specifically ChatGPT for non-substantive editorial assistance in refining the manuscript's language, grammar, and structural flow.

References

[1] anonymous, "Large Language Models in Generating UML Class Diagram," GitHub repository, 2025. [Online]. Available: https://anonymous.4open.science/r/Large-Language-Models-in-Generating-UML-Class-Diagram-785E

[2] M. Barenkamp, J. Rebstadt, and O. Thomas, "Applications of AI in Classical Software Engineering," *AI Perspectives & Advances*, vol. 2, no. 1, 2020, doi: 10.1186/s42467-020-00005-4. [Online]. Available: https://aiperspectives.springeropen.com/articles/10.1186/s42467-020-00005-4

[3] İ. Özkaya, "Application of large language models to software engineering tasks: Opportunities, risks, and implications," *IEEE Software*, vol. 40, no. 3, pp. 4–8, 2023, doi: 10.1109/MS.2023.3248401.

[4] M. Waseem, T. Das, A. Ahmad, P. Liang, M. Fehmideh, and T. Mikkonen, "ChatGPT as a Software Development Bot: A Project-Based Study," in *Proc. 19th Int. Conf. Evaluation of Novel Approaches to Software Engineering (ENASE)*, Feb. 2024, doi: 10.5220/0012631600003687. Preprint: arXiv:2310.13648 (Oct. 2023).

[5] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements Elicitation, and Software Design," *arXiv preprint arXiv:2303.07839*, 2023.

[6] D. Rouabhia and I. Hadjadj, "Behavioral Augmentation of UML Class Diagrams: An Empirical Study of Large Language Models for Method Generation," *arXiv preprint arXiv:2506.00788*, 2025.

[7] B. Al-Ahmad, A. Alsobeh, O. Meqdadi, and N. Shaikh, "A Student-Centric Evaluation Survey to Explore the Impact of LLMs on UML Modeling," *Information*, vol. 16, no. 7, p. 565, 2025, doi: 10.3390/info16070565.

[8] K. Chen, Y. Yang, B. Chen, J. A. Hernández López, G. Mussbacher, and D. Varró, "Automated Domain Modeling with Large Language Models: A Comparative Study," in *Proc. 26th ACM/IEEE Int. Conf. Model Driven Eng. Lang. Syst. (MODELS)*, Västerås, Sweden, Oct. 2023, pp. 162–172, doi: 10.1109/models58315.2023.00037.

[9] A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, M. S. Aktar, and T. Mikkonen, "Towards Human-Bot Collaborative Software Architecting with ChatGPT," in *Proc. 27th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 279–285, doi: 10.1145/3593434.3593468.

[10] D. Russo, "Navigating the Complexity of Generative AI Adoption in Software Engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, pp. 1–50, Jun. 2024, doi: 10.1145/3652154. [Online]. Available: https://dl.acm.org/doi/10.1145/3652154

[11] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022.

[12] S. Bubeck *et al.*, "Sparks of Artificial General Intelligence: Early Experiments with GPT-4," *arXiv preprint arXiv:2303.12712*, 2023.

[13] H. Pearce *et al.*, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *IEEE Symposium on Security and Privacy*, 2023.

[14] X. Wang *et al.*, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *arXiv preprint arXiv:2203.11171*, 2023.

[15] Anonymous, "Evaluating and Enhancing Large Language Models in Generating UML Class Diagram for Good Code Design," in *Women in Machine Learning Workshop @ NeurIPS*, 2025.

[16] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. Pinto, J. Kaplan, ..., and W. Zaremba, "Evaluating Large Language Models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[17] A. Tagliaferro, S. Corboe, and B. Guindani, "Leveraging LLMs to Automate Software Architecture Design from Informal Specifications," in *Proc. 2025 IEEE 22nd Int. Conf. Softw. Archit. Companion (ICSA-C)*, pp. 291–299, IEEE, 2025. doi: 10.1109/ICSA-C2025.

[18] H. Cervantes, R. Kazman, and Y. Cai, "An LLM-Assisted Approach to Designing Software Architectures Using ADD," *arXiv preprint arXiv:2506.22688*, 2025.

[19] M. Ojha, S. Gupta, and R. Sharma, "Towards Class Diagram Generation from User Stories Using LLMs," in *Proc. 2025 Int. Conf. Next Generation Information System Engineering (NGISE)*, vol. 1, IEEE, 2025. doi: 10.1109/NGISE2025.

[20] M. Ben Chaaben, L. Burgueño, and H. Sahraoui, "Towards Using Few-Shot Prompt Learning for Automating Model Completion," in *Proc. 45th Int. Conf. Software Engineering: New Ideas and Emerging Results (ICSE-NIER'23)*, Sep. 2023, IEEE/ACM, doi: 10.1109/ICSE-NIER58687.2023.00008. Preprint available on arXiv:2212.03404 (Dec. 7, 2022).

[21] R. Chen, J. Shen, and X. He, "A Model Is Not Built By A Single Prompt: LLM-Based Domain Modeling With Question Decomposition," Preprint, arXiv:2410.09854, 2024. Submitted Oct. 13, 2024.

[22] M. F. Wong and C. W. Tan, "Aligning crowd-sourced human feedback for reinforcement learning on code generation by large language models," *IEEE Transactions on Big Data*, pp. 1–12, 2024, doi: 10.1109/TBDATA.2024.3524104. Preprint available on arXiv:2503.15129.

[23] J. Cámara-Moreno, J. Troya-Castilla, L. Burgueño-Caballero, and A. J. Vallecillo-Moreno, "On the Assessment of Generative AI in Modeling Tasks: An Experience Report with ChatGPT and UML," *Software and Systems Modeling*, vol. 22, no. 3, pp. 781–793, 2023, doi: 10.1007/s10270-023-01105-5. [Online]. Available: https://link.springer.com/article/10.1007/s10270-023-01105-5

[24] H.-E. Eriksson and M. Penker, *Mastering UML with Rational Rose 2002, Vol. 1*. Alameda, CA, USA: Sybex, 2002, ISBN 9780782140604.

[25] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020, ISBN 9781259872976.

[26] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Software Architecture*. Reading, MA, USA: Addison-Wesley, 1995.

[27] A. Hunt and D. Thomas, *The Pragmatic Programmer: From Journeyman to Master*. Boston, MA, USA: Addison-Wesley Professional, 1999, ISBN 9780201616224.

[28] J. Liu, Z. Chen, Y. Wang, *et al.*, "Large language models for software engineering: A survey of evaluations, applications, and challenges," *Frontiers in Computer Science*, vol. 7, p. 1519437, 2025, doi: 10.3389/fcomp.2025.1519437.

[29] J. Cámara, M. Wimmer, and E. Burger, "Large language models for model-driven engineering: Opportunities, challenges, and future directions," *arXiv preprint arXiv:2306.00788*, 2023.

[30] J. Cámara, M. Wimmer, and E. Burger, "Toward standardized benchmarks for large language models in model-driven engineering," *Software and Systems Modeling*, Springer, 2024, doi: 10.1007/s10270-024-01206-9.

[31] Y. Li, J. Keung, X. Ma, C. Y. Chong, J. Zhang, and Y. Liao, "LLM-Based Class Diagram Derivation from User Stories with Chain-of-Thought Promptings," in *Proc. 2024 IEEE 48th Annu. Comput., Softw., and Appl. Conf. (COMPSAC)*, Osaka, Japan, Jul. 2024, pp. 45–50, doi: 10.1109/COMPSAC61105.2024.00017.

[32] A. Della Porta, V. De Martino, G. Recupito, C. Iemmino, G. Catolino, D. Di Nucci, and F. Palomba, "Using Large Language Models to Support Software Engineering Documentation in Waterfall Life Cycles: Are We There Yet?," in *CEUR Workshop Proc., vol. 3762*, 2024, pp. 452–457. Ital-IA Intelligenza Artificiale – Thematic Workshops.

[33] N. Moha, Y.-G. Guéhéneuc, L. Duchien, and A.-F. Le Meur, "Decor: A method for the specification and detection of code and design smells," *IEEE Transactions on Software Engineering*, vol. 36, no. 1, pp. 20–36, 2009.

[34] T. Ahmed, P. Devanbu, C. Treude, and M. Pradel, "Can LLMs replace manual annotation of software engineering artifacts?" in *Proc. 2025 IEEE/ACM 22nd Int. Conf. on Mining Software Repositories (MSR)*, pp. 526–538, Apr. 2025.